

Set 1 - End Semester Lab Exam - Sec. A

Instructions:

1. Attempt **any one** of the questions given below.
2. Use only the following editors: nano, vim, gedit, or nedit.
3. You can write your code in **C/C++**.
4. File naming in this format - **SetNo_QNo_RollNo.c/cpp**, e.g. 1_1_2025125.c, 1_1_2025125.cpp
5. **Google form link for submission:** <https://forms.gle/sV8VeqWEg5tfphAp9>

Question 1: Hashing

The following C program implements a Hash Table of size 10 that uses chaining to resolve collisions (via linked lists at each index). Complete the implementation of the following functions [CO1, CO2, CO3, CO4]:

1. void insert(int key): Insert a key into the hash table.
2. int search(int key): If key found, return its index; else return -1.
3. int deleteKey(int key): If key found and deleted, return 1; else return 0.

Base Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define SIZE 10
4 struct Node {
5     int key;
6     struct Node* next;
7 };
8 struct Node* hashTable[SIZE];
9 void initializeHashTable(){
10     for(int i = 0; i<SIZE; i++)
11         hashTable[i] = NULL;
12 }
13 int h(int key) { // hash function
14     return key%SIZE; // h(k) = k mod m. where m = size of the hash table
15 }
16 void insert(int key) { // Write your code here }
17 int search(int key) { // Write your code here }
18 int deleteKey(int key){ // Write your code here }
19 void display() {
20     for (int i = 0; i < SIZE; i++) {
21         printf("Index %d: ", i);
22         struct Node* temp = hashTable[i];
23         while (temp != NULL) {
24             printf("%d -> ", temp->key);
25             temp = temp->next;
26         }
27         printf("NULL\n");
28     }
29 }
30 int main() {
31     int i, idx, a[] = {10 , 20 , 15 , 7 , 32 , 40, 42, 79, 88};
32     initializeHashTable();
33     for (i = 0; i < 9; i++)
34         insert(a[i]);
35     printf("\nHash Table:\n");
36     display();
37     idx = search(15);
38     if (idx != -1)
39         printf("\nKey 15 found at index %d\n",idx);
40     else
41         printf("Key 15 not found\n");
42     if(deleteKey(20))
43         printf("Key 20 deleted\n");
44     else
45         printf("Key 20 not found for deletion\n");
46     printf("\nHash Table after deletion:\n");
47     display();
48     return 0;
49 }
```

Question 2: Binary Search Tree (BST)

The following C program builds a BST with the keys 15, 6, 18, 3, 7, 17, 20, 2, 4, 13. Complete the implementation of the following functions [CO1, CO2, CO3, CO4]:

1. Node* treeMin(Node* x): Return node with minimum key in subtree.
2. Node* treeSuccessor(Node* x): Return successor of node x.
3. void inorder(Node* x): Print inorder traversal of the BST. Verify your output - it must print keys in sorted order.

Base Code

```

1 #include<stdio.h>
2 #include<stdlib.h>
3 typedef struct Node{
4     int key;
5     struct Node* left, *right, *p;
6 } Node;
7 typedef struct Tree{
8     Node* root;
9 } Tree;
10 Node* newNode(int key){
11     Node* n = (Node*)malloc(sizeof(Node));
12     n->key = key; n->left = NULL; n->right = NULL; n->p = NULL;
13     return n;
14 }
15 void insertNode(Tree* T, int key){
16     Node* z = newNode(key); Node* x = T->root; Node* y = NULL;
17     while (x != NULL){
18         y = x;
19         if (z->key < x->key)
20             x = x->left;
21         else
22             x = x->right;
23     }
24     z->p = y;
25     if (y == NULL)
26         T->root = z;
27     else if (z->key < y->key)
28         y->left = z;
29     else
30         y->right = z;
31 }
32 Node* treeSearch(Node* x, int k){
33     if (x == NULL || k == x->key)
34         return x;
35     if (k < x->key)
36         return treeSearch(x->left, k);
37     else
38         return treeSearch(x->right, k);
39 }
40 Node* treeMin(Node* x){ // Write your code here }
41 Node* treeSuccessor(Node* x){ // Write your code here }
42 void inorder(Node* x){ // Write your code here }
43 int main(){
44     int i, searchKey[] = {13, 17}, arr[] = {15, 6, 18, 3, 7, 17, 20, 2, 4, 13};
45     Tree T;
46     T.root = NULL;
47     for (i = 0; i<10; i++)
48         insertNode(&T, arr[i]);
49     Node* node, *successor;
50     for (i = 0; i<2; i++){
51         node = treeSearch(T.root, searchKey[i]);
52         if (node == NULL){
53             printf("Key %d not found in the tree.\n", searchKey[i]);
54         } else {
55             successor = treeSuccessor(node);
56             if (successor == NULL)
57                 printf("Successor of %d: does not exist.\n", searchKey[i]);
58             else
59                 printf("Successor of %d: %d\n", searchKey[i], successor->key);
60         }
61     }
62     printf("\nInorder traversal: ");
63     inorder(T.root);
64     printf("NULL\n");
65     return 0;
66 }

```

Solution 1

```
1 void insert(int key) {
2     int index = h(key);
3     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
4     newNode->key = key;
5     newNode->next = NULL;
6     //inserting in the beginning
7     newNode->next = hashTable[index];
8     hashTable[index] = newNode;
9 }
10 int search(int key) {
11     // If key found: return index, else return -1
12     int index = h(key);
13     struct Node* temp = hashTable[index];
14     while (temp != NULL) {
15         if (temp->key == key)
16             return index;
17         temp = temp->next;
18     }
19     return -1;
20 }
21 int deleteKey(int key){
22     // If key found and deleted: return 1, else return 0
23     int index = h(key);
24     struct Node* temp = hashTable[index];
25     if (temp != NULL && temp->key == key){
26         hashTable[index] = temp->next;
27         free(temp);
28         return 1;
29     }
30     while (temp != NULL) {
31         if (temp->next != NULL && temp->next->key == key){
32             struct Node* toDelete = temp->next;
33             temp->next = temp->next->next;
34             free(toDelete);
35             return 1;
36         }
37         temp = temp->next;
38     }
39     return 0;
40 }
```

Output

```
1 Hash Table:
2 Index 0: 40 -> 20 -> 10 -> NULL
3 Index 1: NULL
4 Index 2: 42 -> 32 -> NULL
5 Index 3: NULL
6 Index 4: NULL
7 Index 5: 15 -> NULL
8 Index 6: NULL
9 Index 7: 7 -> NULL
10 Index 8: 88 -> NULL
11 Index 9: 79 -> NULL
12
13 Key 15 found at index 5
14 Key 20 deleted
15
16 Hash Table after deletion:
17 Index 0: 40 -> 10 -> NULL
18 Index 1: NULL
19 Index 2: 42 -> 32 -> NULL
20 Index 3: NULL
21 Index 4: NULL
22 Index 5: 15 -> NULL
23 Index 6: NULL
24 Index 7: 7 -> NULL
25 Index 8: 88 -> NULL
26 Index 9: 79 -> NULL
```

Rubric

Criteria	Marks
Correct node creation and insertion into the hash table using chaining	2
Correct hash index computation and traversal in <code>search()</code>	2
Correct traversal and pointer updates in <code>deleteKey()</code> with memory free	3
Handles edge cases (E.g., head deletion, key not found)	1
Correct output	2
Total	10

Solution 2

```

1 Node* treeMin(Node* x){
2     while (x->left != NULL)
3         x = x->left;
4     return x;
5 }
6 Node* treeSuccessor(Node* x){
7     if (x->right != NULL)
8         return treeMin(x->right);
9     Node* y=x->p;
10    while (y !=NULL && x ==y->right){
11        x = y;
12        y = y->p;
13    }
14    return y;
15 }
16 void inorder(Node* x){
17     if (x == NULL)
18         return;
19     inorder(x->left);
20     printf("%d -> ",x->key);
21     inorder(x->right);
22 }

```

Output

```

1 Successor of 13: 15
2 Successor of 17: 18
3
4 Inorder traversal: 2 -> 3 -> 4 -> 6 -> 7 -> 13 -> 15 -> 17 -> 18 -> 20 -> NULL

```

Rubric

Criteria	Marks
Correct implementation of <code>treeMin()</code>	2
Correct implementation of <code>treeSuccessor()</code> with parent pointer traversal	1+3 (parent traversal) = 4
Correct implementation of <code>inorder()</code> traversal	2
Correct output	2
Total	10